

Test Plan

SPI Master Controller — Digital Design Verification

Spring 2026 Final Project

Document Version: 1.0

Date: May 6, 2026

Methodology: SystemVerilog (Plain SV)

Simulator: QuestaSim

Table of Contents

1. Introduction
2. Test Environment Overview
3. Test Case Summary
4. Detailed Test Cases (TC-01 through TC-10)
5. Regression Strategy
6. Pass/Fail Criteria & Log Contract

1. Introduction

This document defines the test plan for verifying the SPI Master Controller IP. The plan maps directly to the 10 mandatory test names required by the grading interface contract (Section 3) and covers functional requirements R1-R25 from the design specification.

Each test case specifies its purpose, targeted requirements, stimulus strategy, expected results, and pass/fail criteria. Tests are dispatched via `+TESTNAME=<name>` from the `tb_top` module and follow the class-based static `task run()` pattern.

2. Test Environment Overview

Component	File	Role
Top-level module	tb/tb_top.sv	Clock/reset gen, DUT wrapper, BFM instances, test dispatcher
APB Master BFM	tb/apb_master_bfm.sv	Drives APB read/write transactions to configure & access the DUT
SPI Slave BFM	tb/spi_slave_bfm.sv	Emulates downstream SPI slave; drives MISO, captures MOSI
Reference Model	env/ref_model.sv	Golden predictor for registers, FIFOs, SPI data, and interrupts
Coverage Collector	env/coverage.sv	Functional covergroups (mode, width, div, FIFO, IRQ, delay)
Stimulus Library	sequences/stim_lib.sv	Randomisable transaction classes with constraints
SVA Assertions	assertions/spi_sva.sv	Bound to DUT internals for continuous protocol checking

3. Test Case Summary

#	Test Name	Requirements	Category
1	sanity_test	R1-R7	Smoke / Connectivity
2	reg_access_test	R1, R2	Register Interface
3	mode_coverage_test	R4, R5, R6, R7	SPI Protocol
4	width_coverage_test	R6, R7	Data Path
5	fifo_stress_test	R9, R10, R11, R12	FIFO Management
6	interrupt_test	R16, R17, R18	Interrupt Controller
7	clk_div_corner_test	R8, R24, R25	Clock Generation
8	loopback_test	R19	Advanced Mode
9	delay_transfer_test	R21	Timing / Delay
10	error_injection_test	R13, R14, R15, R23	Robustness

4. Detailed Test Cases

TC-01: sanity_test

Purpose	Verify the most basic operational sequence to ensure the testbench and DUT are alive and connected.
Requirements	R1-R7
Scope	Enable the controller, execute a single byte (8-bit) transfer in SPI Mode 0, and verify the received data.
Stimulus	1. Assert PRESETn low for ≥ 2 PCLK cycles, then release. 2. Write CTRL = 0x03 (EN=1, MSTR=1, Mode 0, MSB-first, 8-bit). 3. Write CLK_DIV = 0x04. 4. Write TX_DATA = 0x5A; assert SS_CTRL = 0x01. 5. Poll STATUS.BUSY until deasserted. 6. Read RX_DATA.
Expected Results	SCLK idles low; data shifts correctly; RX_DATA matches the expected slave response (0xA5 from the BFM pattern).
Pass/Fail	Pass if single byte transmitted and received correctly. Fail if the transaction hangs or data is corrupted.

TC-02: reg_access_test

Purpose	Verify the APB slave interface reliability and reset behavior.
Requirements	R1 R2
Scope	Check reset values for all 9 registers, followed by a write-read sequence for every R/W register.
Stimulus	1. After reset, read all 9 registers and compare to spec defaults. 2. Write a known pattern to each R/W register (CTRL, TX_DATA, CLK_DIV, SS_CTRL, INT_EN, DELAY). 3. Read back and verify masked values. 4. Write to reserved offsets (0x24, 0x28, ...), read back → must return 0. 5. Verify RO registers (STATUS, RX_DATA, INT_STAT) ignore writes.
Expected Results	All registers default to specified reset values; reads return exactly the last written value (masked by RO bits).
Pass/Fail	Pass if all backdoor and frontdoor checks match. Fail if any bit is stuck or alters unintentionally.

TC-03: mode_coverage_test

Purpose	Verify all major SPI operational modes.
Requirements	R4 R5 R6 R7
Scope	Cross-coverage of all 4 SPI modes (CPOL/CPHA) × {MSB-first, LSB-first} × {8, 16, 32 bits} = 24 combinations.
Stimulus	For each of the 24 combinations: 1. Configure CTRL with the target mode, ordering, and width. 2. Synchronise the SPI Slave BFM to the same mode. 3. Push a known TX_DATA pattern, assert SS, wait for completion. 4. Read RX_DATA and compare against prediction.
Expected Results	SCLK idle polarity and sample/launch edges adjust dynamically; bit ordering is strictly followed.
Pass/Fail	Pass if slave BFM accurately decodes all 24 payloads. Fail if MOSI transitions on the wrong edge.

TC-04: width_coverage_test

Purpose	Verify edge cases at transfer width boundaries.
Requirements	R6 R7
Scope	Back-to-back testing of 8-bit, 16-bit, and 32-bit transfers.
Stimulus	1. Configure for 8-bit transfer, send data, verify SCLK count = 8. 2. Reconfigure for 16-bit, send data, verify SCLK count = 16. 3. Reconfigure for 32-bit, send data, verify SCLK count = 32. 4. Test width transitions back-to-back without disabling the controller.
Expected Results	Transfers take exactly WIDTH SCLK cycles; BUSY deasserts exactly one PCLK after the last sample edge.
Pass/Fail	Pass if no bits truncated or extended. Fail if APB interface misaligns the data payload.

TC-05: fifo_stress_test

Purpose	Verify TX and RX FIFO pointers under heavy load without triggering overflows.
Requirements	R9 R10 R11 R12
Scope	Push/pop operations near full/empty conditions and continuous back-to-back transfers keeping the pipeline saturated.
Stimulus	1. Fill TX FIFO with 8 entries; verify TX_FULL asserts. 2. Start transfers; read RX_DATA as each completes. 3. Verify RX_FULL asserts when 8 entries are received without reads. 4. Run continuous back-to-back transfers to stress pointer wrap-around.
Expected Results	TX_FULL asserts at exactly 8 entries; data shifts continuously without dropping elements.
Pass/Fail	Pass if FIFO ordering is strictly FIFO. Fail if data sequence corrupted or pointers mismatch.

TC-06: interrupt_test

Purpose	Verify the complete interrupt lifecycle and W1C mechanisms.
Requirements	R16 R17 R18
Scope	Each of the 5 interrupts: TRANSFER_DONE, RX_OVF, TX_OVF, RX_FULL, TX_EMPTY — trigger, mask, W1C, W1C race.
Stimulus	1. Enable all interrupt masks; trigger TRANSFER_DONE by completing a transfer → verify IRQ=1. 2. W1C clear the flag → verify IRQ=0. 3. Mask one source; trigger it → verify INT_STAT set but IRQ=0. 4. Trigger a hardware event on the same PCLK cycle as a W1C write → verify bit stays set. 5. Repeat for each of the 5 sources.
Expected Results	Masked events set INT_STAT but hold IRQ=0; W1C race preserves the bit at 1.
Pass/Fail	Pass if $IRQ = (INT_STAT \& INT_EN)$. Fail if events are dropped during W1C race.

TC-07: clk_div_corner_test

Purpose	Verify the SCLK generation logic at corner-case divider values.
Requirements	R8 R24 R25
Scope	DIV = 0, 1, small random values, and large values (≥ 1024) up to 65535.
Stimulus	1. For each DIV value: configure CLK_DIV, start a transfer, measure the SCLK period. 2. Compare measured period against $PCLK/(2 \times (DIV+1))$. 3. Verify 50% duty cycle on SCLK.
Expected Results	$SCLK \text{ frequency} = PCLK/(2 \times (DIV+1))$; DIV=0 yields $PCLK/2$ without errors.
Pass/Fail	Pass if measured period matches formula exactly. Fail if duty cycle deviates from 50%.

TC-08: loopback_test

Purpose	Verify the internal self-test routing functionality.
Requirements	R19
Scope	CTRL.LOOPBACK = 1; external MISO driven with nonsense data by the slave BFM.
Stimulus	1. Set CTRL.LOOPBACK=1, configure mode 0, 8-bit. 2. Drive MISO with 0xFF (nonsense) from the slave BFM. 3. Push TX_DATA = 0x5A, start transfer. 4. Read RX_DATA → should be 0x5A (looped from TX), NOT 0xFF.
Expected Results	MOSI is routed internally to RX; nonsense MISO data is ignored completely.
Pass/Fail	Pass if RX_DATA == TX_DATA. Fail if external MISO pollutes the RX FIFO.

TC-09: delay_transfer_test

Purpose	Verify the inter-transfer delay logic.
Requirements	R21
Scope	Set DELAY > 0 and queue multiple transfers in the TX FIFO.
Stimulus	1. Configure DELAY = 5, push 3 entries to TX FIFO. 2. Assert SS and start transfers. 3. Monitor SCLK: count idle half-cycles between consecutive transfers. 4. Verify BUSY stays asserted during the delay gap.
Expected Results	The master inserts exactly DELAY SCLK half-cycles of idle time between consecutive transfers while holding BUSY=1.
Pass/Fail	Pass if gap matches the programmed delay precisely. Fail if delay is skipped or applied incorrectly.

TC-10: error_injection_test

Purpose	Verify the controller's robustness against illegal software sequences.
Requirements	R13 R14 R15 R23
Scope	Write to TX_DATA when full, read RX_DATA when empty, illegal width encoding (2'b11), reserved register offsets (0x24+).
Stimulus	1. Fill TX FIFO (8 writes), then write once more → verify TX_OVF flag. 2. Read RX_DATA without any transfers pending → verify returns 0, no RX_OVF. 3. Write CTRL.WIDTH = 2'b11 (illegal) → observe behavior (undefined, no crash). 4. Write/read reserved offsets 0x24, 0x28, 0x2C → writes ignored, reads return 0.
Expected Results	Overflow flags trigger on full bounds; empty RX reads return 0 without triggering overflow; reserved offsets are inert.
Pass/Fail	Pass if IP safely discards illegal accesses and sets appropriate flags. Fail if state machine deadlocks or asserts PSLVERR.

5. Regression Strategy

Parameter	Value
Total required tests	10
Seeds per test	20 (configurable via REGRESSION_SEEDS)
Total simulation runs	200 (well under the 10,000 cap)
Timeout per test	10 minutes wall-clock
Determinism	All tests are reproducible given +SEED=<n>

6. Pass/Fail Criteria & Log Contract

Every test must end with exactly one of the following lines:

- [TEST_PASSED] <test_name> — zero errors, all checks passed
- [TEST_FAILED] <test_name> errors=<n> — at least one error detected

Bug detection is signalled via:

- [SCOREBOARD_ERROR] <description> — data mismatch
- [ASSERTION_ERROR] <name> <description> — SVA failure
- [CHECKER_ERROR] <name> <description> — explicit \$error check

